

Memoria Completa de Arquitectura RISC-V (RV32I)

1. Estructura de un Programa en Ripes

Cualquier programa en ensamblador se organiza en dos secciones fundamentales:

- **.data**: Bloque reservado para la memoria estatica. Aqui declaramos las variables antes de dar paso a la ejecucion del codigo.
 - **.word**: Asigna 32 bits (4 bytes) para guardar un entero. (Ej: array: .word 1, 2, 3)
 - **.string**: Reserva espacio para una cadena de texto y añade de forma automática el carácter nulo \0 al final.
- **.text**: Bloque que contiene las instrucciones del programa. Debe empezar siempre con la etiqueta main: (o la que se defina como punto de entrada).

2. Convención de Registros (ABI)

Aunque el procesador dispone de 32 registros, la convención establece un uso específico para evitar fallos de ejecución:

- **zero (x0)**: Mantiene un valor constante de 0. Resulta muy practico para limpiar o poner a cero otros registros (mv t0, zero).
- **ra (x1)**: *Return Address*. La instrucción jal guarda aquí de forma automática la dirección a la que hay que volver tras llamar a una función.
- **sp (x2)**: *Stack Pointer*. Apunta al extremo superior de la pila. **Ojo**: la pila crece hacia direcciones de memoria más bajas, así que para reservar espacio hay que **restar**.
- **t0 - t6**: Registros temporales. Se pueden usar para operaciones intermedias, bucles o manejo de punteros. Ten en cuenta que su valor no se conserva al hacer llamadas a funciones.
- **a0 - a7**: Se emplean para pasar parámetros a las funciones y para gestionar las llamadas al sistema (ecall).
- **s0 - s11**: Registros preservados. Si los utilizas dentro de una función, estás obligado a guardar su contenido previo en la pila y restaurarlo antes de regresar.
- **gp (x3)**: Global Pointer. Sirve para apuntar a la zona de datos estáticos (.data) y facilitar un acceso rápido a las variables globales.

3. Formatos de Instrucción y Pseudo-instrucciones (Añadido)

Las instrucciones en RISC-V se estructuran segun el formato con el que se codifican en binario. Para el examen conviene dominar cada tipo:

- **Tipo R (Registro)**: Para operaciones aritméticas y lógicas en las que solo intervienen registros (Ej: add, and, xor, slt).
- **Tipo I (Inmediato)**: Trabajan combinando un registro y una constante o valor inmediato (Ej: addi, andi, lw, jalr).

- **Tipo S (Store):** Destinadas a volcar información en la memoria (Ej: sw).
- **Tipo B (Branch):** Saltos condicionales que dependen del resultado de una comparación (Ej: beq, bne, blt, bge).
- **Tipo U (Upper Immediate):** Sirven para cargar valores inmediatos en los bits más significativos del registro (Ej: lui, auipc).
- **Tipo J (Jump):** Saltos incondicionales relativos a la posición actual del PC (Ej: jal).

Pseudo-instrucciones: Son atajos que nos ofrece el ensamblador y que luego se traducen internamente en instrucciones nativas más básicas. Imprescindibles para el examen:

- `nop` → equivale a `addi x0, x0, 0`
- `mv rd, rs` → equivale a `addi rd, rs, 0`
- `li rd, inmm` → según el tamaño del dato, se transforma en `lui` o en la combinación `lui + addi`.
- `j offset` → equivale a `jal x0, offset`

A tener en cuenta con las multiplicaciones: El juego de instrucciones base RV32I no trae la operación `mul`. Para calcular un producto (por ejemplo, $3 \cdot x$) hay que recurrir a sumas consecutivas: primero `add t4, t3, t3` (obteniendo $2x$) y a continuación `add t4, t4, t3` (obteniendo $3x$).

4. Llamadas al Sistema Críticas (ecall)

Para indicarle a Ripes que interactue con la consola o la salida del sistema, cargamos el código del servicio correspondiente en el registro `a7`:

- **Imprimir Entero:** `li a7, 1` (teniendo el entero preparado en `a0`).
- **Imprimir Cadena:** `li a7, 4` (pasando en `a0` la dirección donde empieza el string).
- **Finalizar Programa (IMPRESINDIBLE):** `li a7, 10`. Si te olvidas de incluir esta llamada al terminar `main`, la CPU seguirá procesando posiciones vacías de memoria hasta lanzar una excepción.

5. Gestión de Memoria y Punteros (El error más común)

En RISC-V las direcciones de memoria van referenciadas byte a byte, pero hay que recordar que una palabra completa (`.word`) ocupa **4 bytes**.

- **Acceso a un vector:** Si `t0` apunta al inicio del vector de enteros, el índice 0 se lee con `lw t1, 0(t0)`. Para pasar al índice 1 **debemos sumar 4 bytes, no 1**: `lw t1, 4(t0)`.
- **Avanzar un puntero en bucles:** Dentro de una iteración hay que desplazar la dirección base sumando 4 con `addi t0, t0, 4`.

6. Plantillas Definitivas de Control de Flujo

Bifurcación (If-Else)

En ensamblador la lógica funciona de forma invertida respecto a lenguajes de alto nivel. Si la condición para ejecutar un bloque es $x > 0$, en la práctica le indicamos al procesador: "si el valor es menor o igual a 0, salta y márchate al else".

Suponemos que el dato a probar está en `t1`

```

ble t1, zero, caso_else # Si t1 <= 0, saltamos directamente al else
caso_if:
# Código que se ejecuta cuando t1 > 0
j fin_if # Salto para no meternos en el bloque else
caso_else:
# Código que se ejecuta cuando t1 <= 0
fin_if:

```

Bucle Recorriendo un Array

```

la t0, array # t0 = Puntero base del array
li t1, 0 # t1 = Contador de iteraciones (i = 0)
li t2, 8 # t2 = Tamaño máximo del array (8 elementos)
loop:
beq t1, t2, end # Condición de salida: si i == 8, terminar bucle
lw t3, 0(t0) # Carga el elemento actual en t3
# ... operaciones con t3 ...
addi t0, t0, 4 # CRÍTICO: Avanzar puntero 4 bytes
addi t1, t1, 1 # Avanzar contador (i++)
j loop # Volver a evaluar la condición
end:

```

Funciones y Uso de la Pila (Stack)

Siempre que una función llame a otra subrutina o haga uso de los registros s, es indispensable apoyarse en la pila para preservar los datos.

```

mi_funcion:
# 1. Prólogo (Guardar contexto)
addi sp, sp, -16 # Abrimos espacio para 4 palabras (4 * 4 bytes)
sw ra, 12(sp) # Salvamos la dirección de retorno
sw s0, 8(sp) # Salvamos los registros protegidos que vayamos a
usar

# 2. Cuerpo de la función
# ... instrucciones ...

# 3. Epílogo (Restaurar contexto)
lw ra, 12(sp) # Recuperamos la dirección de retorno original
lw s0, 8(sp)
addi sp, sp, 16 # Liberamos el espacio de la pila (sumando)
jr ra # Regresamos al punto donde se nos llamó

```

Bucle con Doble Puntero y Acumulador (Lectura, Escritura y Suma)

Ejemplo para leer datos de un arreglo, procesarlos, volcarlos en un segundo arreglo y acumular

una suma total:

```
la t0, array_in      # Puntero para la lectura
la t1, array_out      # Puntero para la escritura
li t2, 0              # Índice i=0
li t3, 8              # Límite del bucle
li t4, 0              # Acumulador para la suma
loop:
beq t2, t3, end
lw t5, 0(t0)          # Lectura del dato
# ... procesamiento ...
sw t6, 0(t1)          # Escritura del resultado
add t4, t4, t6         # Suma acumulada
addi t0, t0, 4         # Desplazamos puntero de lectura
addi t1, t1, 4         # Desplazamos puntero de escritura
addi t2, t2, 1         # Incrementamos contador
j loop
end:
```

7. Rendimiento y Procesador Segmentado (Pipeline)

Conceptos Clave

- **Uniciclo:** Cada instrucción requiere 1 ciclo de reloj. Trabaja a menor frecuencia pero mantiene el CPI ideal (CPI=1).
- **Segmentado (Pipeline de 5 etapas):** IF (Fetch), ID (Decode), EX (Execute), MEM (Memory) y WB (Write-back). Superpone la ejecución de varias instrucciones para elevar la frecuencia de reloj.
- **Riesgo de Datos (Data Hazard):** Ocurre si una instrucción intenta leer un registro cuyo valor aún está siendo procesado por una instrucción previa.
- **Adelantamiento (Forwarding):** Circuito de hardware que reencamina los datos calculados para no perder ciclos esperando. En Ripes verás dos variantes:
 - **EX → EX:** La salida de la ALU pasa de forma inmediata a la entrada de la ALU en el siguiente ciclo.
 - **MEM/WB → EX:** El dato recuperado de la memoria se dirige directamente a la etapa de ejecución de la instrucción posterior.

Cálculo del Rendimiento, CPI e IPC

La arquitectura más eficiente será siempre la que reduzca al mínimo el tiempo final obtenido mediante esta relación:

$$\text{Tiempo} = (\text{Instrucciones} \times \text{CPI}) / \text{Frecuencia}$$

- En un diseño **monociclo**, el valor de CPI (Ciclos Por Instrucción) equivale exactamente a **1**.
- En una arquitectura **segmentada** con paradas por riesgos de datos, el CPI acabará siendo **superior a 1** (por ejemplo 1.29 o 1.77).
- Un procesador con pipeline puede tardar menos tiempo total que un monociclo aunque

su CPI sea mayor, ya que trabaja a una frecuencia de reloj muy superior que compensa esos ciclos extra.

Regla de Oro de los NOPs en Ripes (Sin Adelantamiento)

Si la prueba requiere trabajar con el procesador **5-Stage w/o Forwarding**, habrá que añadir instrucciones nulas (nop) a mano para que no se lean valores desactualizados.

- **Situación típica de conflicto:** Una instrucción escribe en un registro (ej: addi, li) e inmediatamente la siguiente intenta leerlo (ej: sw, bne).
- **Regla en Ripes:** Se exigen **3 ciclos de diferencia** entre el momento en que se escribe el dato y cuando se lee. Si van seguidas, metemos 3 nop. Si hay una instrucción útil intercalada, añadimos 2 nop. Si hay dos útiles, basta con 1 nop.

El Riesgo "Load-Use" y Optimización por Reordenación

El riesgo **Load-Use** aparece en el momento en que una instrucción lw lee de memoria e **inmediatamente la siguiente** necesita utilizar ese valor cargado (por ejemplo, en un beq, slt o add).

Incluso teniendo activo el mecanismo de adelantamiento por hardware, una situación Load-Use causa inevitablemente 1 ciclo de parada (stall) en Ripes.

Optimización por reordenamiento: Para deshacernos de esa penalización sin meter NOPs, cambiamos el orden de las instrucciones. La técnica consiste en mover una instrucción independiente (como addi t1, t1, 1 o addi t0, t0, 4) justo entre el lw y la instrucción que va a usar el dato. De este modo amortiguamos la espera procesando trabajo útil sin modificar la lógica final.

Caso de Reordenamiento Eficiente: Ineficiente (provoca Stall): poner lw t5, 0(t0) pegado a add t4, t5, t5 (pierde 1 ciclo). Eficiente (sin Stall): meter lw t5, 0(t0), seguido de addi t0, t0, 4 (instrucción independiente que aprovecha ese ciclo) y finalmente add t4, t5, t5.

8. Verificación de Errores Comunes de Examen

1. **Errores de alineación en memoria (Misaligned Address):** Ocurren si se intenta ejecutar lw o sw en direcciones que no son múltiplos de 4. Revisa bien los incrementos addi t0, t0, 4.
2. **Bucles que no terminan:** Comprueba que no se te haya pasado meter el incremento del contador addi t1, t1, 1 o el salto de retorno j loop al cerrar la estructura.
3. **Despistes en saltos condicionales:** Recuerda que si el enunciado en C indica if (x == 5), en ensamblador saltamos para evitar el bloque mediante bne (si NO es 5, se salta).
4. **Pila descompensada:** Si restas -16 a sp en el prólogo, asegúrate de sumar +16 al salir. Dejar la pila desajustada provocará un fallo al intentar volver con jr ra a una dirección errónea.